



# NEURAL NETWORKS

## Image Classification

### Abstract

This report details the decisions made to train a neural network model for the Fashion-MNIST dataset and the findings from the training, evaluated only on the validation dataset.

Tracey Letlape  
ltltra001@myuct.ac.za

## 1. Data Processing

For data processing, I used the **torchvision** library for data extraction and for transforming data.

I used the **transforms** module from the **torchvision** library to define a transforms object that can be used to transform data. The transforms object is defined as follows:

- Defined using the **Compose** class from the transforms module to enable me to compose several transforms together.
- This is very efficient as it allows us to transform data using one step.
- The different transforms are passed to the Compose class as a list and they are as follows:
  - o **transforms.ToTensor()**: to convert PIL Image to tensor
  - o **transforms.Normalize((0.5,), (0.5))**: to normalise the data using standardisation to 0.5 mean and a standard deviation of 0.5.
  - o **transforms.Lambda(lambda x: x.flatten())**: to flatten the image to 1D.

I used the **datasets** module from the **torchvision** library to download the FashionMNIST train and test datasets. Again, this was way efficient as it reduced extra computation needed to convert raw data into a **TensorDataset**. The transforms object defined as stated above was then passed as an argument to the **FashionMNIST** constructor. Very efficient as this step allows us to perform the following in one step:

- Get raw FashionMNIST data
- Transform the data using the above transforms functions.
- Convert data into a TensorDataset.

I used the **random\_split** function from the **torch.util.data** module with a torch generator object of seed 2 to split the training data into training and validation datasets.

Since the test data is 10000 images, I split the training data such that the validation set contains 10000 images.

To reduce the extra time to copy the data during training whenever data is passed as an argument, I created a global variable **\_LOADERS** to store the train, validation and test data loaders. This allows the objects to be used everywhere across the notebook without the need for copying as they are passed as parameters.

## 2. Building and Training a Baseline Model

Defined the following classes and methods to help with model definition and model training:

- **MultiLayerPerceptron**:
  - o The class inherits from the **nn.Module** class, with its constructor taking in four arguments: the number of features of the input data, the number of neurons in the first and second hidden layers respectively, and the number of classes.
  - o The class uses the **nn.Linear** module to compute logits for each layer and uses the **nn.ReLU** activation function for the first and second layer to account for non-linearity as the FashionMNIST dataset is not linearly separable.
  - o The class has the **forward** method which does forward propagation for a batch of inputs and a **predict** method that predicts the classes of the batch of inputs as the classes that maximises the logits.

- **set\_deterministic\_seed** function:
  - o to set seeds for all libraries that are used that generates things randomly to ensure deterministic behaviour.
- **train\_model** function:
  - o that trains the model, keeps track training and validation losses, keeps track of training and validation accuracy, returns a dictionary of the tracked metrics and the trained model.
- **get\_final\_metrics** function:
  - o to compute the final training and validation accuracy from the trained MultiLayerPerceptron model.

Defined a baseline model to train. The baseline **MultiLayerPerceptron** model assumes that data has 784 features (28x28 image pixels), 128 neurons for the first hidden layer, 64 neurons for the second hidden layer and 10 classes. Trained the model with the loss function as the **nn.CrossEntropyLoss()** and the optimiser as **t.optim.Adam()** with the learning rate = 0.001.

The final training accuracy achieved was around 92% and the final validation accuracy was around 88%. This shows that the model is the right model for the dataset, and it learns the underlying patterns extremely well. The training results can be found [here](#).

### 3. Hyperparameter Optimisation Experiment

Defined the **grid\_search** method that takes in a list of learning rates, list of optimisers, and a list of neurons to implement a grid search to find the optimum hyperparameters (the ones that maximises the validation accuracy). The set of learning rates are [0.0001, 0.001, 0.01, 0.1], the set of optimisers are ["Adam", "SGD", "RMSprop"] and the set of neurons are [100, 150, 200, 250]. The models are defined with the **nn.CrossEntropyLoss()** function as the loss function. The **grid\_search** function returns a list of training results for the hyperparameter combinations.

### 4. Analysis and Report

To visualise the results of the **grid\_search**, I used the pandas **dataframe** to view the results in a table format. The **dataframe** was sorted in descending order of the validation accuracy. I also used a matplotlib **barplot** to visualise the results graphically, making it easier to spot the hyperparameter combinations that maximise validation accuracy. The hyperparameter combinations are on the y-axis with the validation accuracy on the x-axis.

To make the comparison of the baseline model and the model that maximises the validation accuracy easier, I used a line graph to visualise the training and validation losses of the two models, the training and validation accuracies of the two models across different epochs on one plot.

Based on the grid results, I identified the best hyperparameter combinations as learning rate=0.001, optimiser="Adam" and the neurons=200. Defined a new MultiLayerPerceptron model on these parameters, trained the model, predicted the classes for the validation dataset and plotted the confusion matrix of the true y-values vs the predicted y-values using a seaborn heatmap.

#### *Interpretation of results:*

**Q: Which combination of hyperparameters produced the best model? How much did it improve upon the baseline accuracy?**

- The combination of hyperparameters that produced the best model was learning rate = 0.001, optimiser = t.optim.Adam and the number of neurons of the first hidden layer was 200.
- The improvement was not that significant, as the validation accuracy improved to 0.8850 from the baseline validation of 0.8833.

**Q: Which hyperparameter had the most influence on performance? Explain why you think certain values worked better than others.**

- The optimiser. Almost all hyperparameter combinations with t.optim.Adam as the optimiser produced the validation accuracies around 80s, while all the other optimisers produced accuracies below 10%.
- Adam's core mechanism is its ability to compute adaptive learning rates for each individual parameter in the neural network, allowing it to navigate the loss landscape more effectively and typically converge faster than standard Stochastic Gradient Descent (SGD).
- Adam goes well with lower learning rates (like  $\leq 0.01$ ) as it allows it to make steady jumps towards the steepest gradient and faster convergence, but not too high like 0.1 as the algorithm will take huge steps towards the steepest gradient, potentially missing local minimum.

**Q: By comparing the loss curves of your baseline and best models, analyse the degree of overfitting. Did your hyperparameter tuning help mitigate overfitting. How can you tell from the plot.**

- The degree of overfitting is not that large for both models. At most, the model's difference between the training and validation losses is 0.1.
- The models start overfitting at around the 5<sup>th</sup> epoch.
- The best model's training loss was consistently lower than that of the baseline model for all epochs.
- However, the best model seemed to increase the degree of overfitting instead of mitigating it.
- We can tell this from the plot, as the difference of the height between the train and validation loss of the baseline model is smaller than the difference of the height between the train and validation loss of the best model.

**Q: Look at you confusion matrix. Identify two classes that the model frequently confuses. Propose a reason for this confusion by considering the visual characteristics of the characteristics of the items in those classes.**

- The model is mostly accurate with its prediction, but there are two pairs of classes the model mostly confuses: class 4 and class 2, and class 6 and class 0.
- This is supported by the large number of misclassifications between these pairs as compared to the misclassification of other classes.
- After visualisation, it becomes a probable reason that these confusions arise from the similar characteristics of the items in these classes.
- Class 0 seems to represent short sleeve round neck shirts while class 6 seems to represent short sleeve V-neck shirts. The difference is subtle and hence why the model sometimes confuses these two classes.

- Class 2 on the other hand seems to represent hoodies while class 4 seems to represent hoodless jackets. Again, the difference is subtle and might be the reason why the model sometimes confuses these two classes.

The results for hyperparameter tuning, the tabular results can be found [here](#) while the rest (graphs, barplots and confusion matrix) can be found [here](#).

The results visualising items represented by each class can be found [here](#).

In conclusion, the model was well suited for the dataset, and performance was great for the baseline model, and subtle improvement after tuning.